

* 이미지 모델 흐름

단계	위치 (파일:함수)	입력	처리	출력
1	pill_chatbot_front / ChatComposer.jsx : handleSend	사용자 텍스트/이미지	이미지 미리보기 URL 생성, 상태 저장	{ text, imageFile, imageUrl }
2	pill_chatbot_front / ChatPage.jsx : handleUserInput	{ text, imageFile, imageUrl }	이미지 유무 확인 → 이미지 API 호출 결정	imageApi.chatPillImage 호출
3	pill_chatbot_front / api/imageApi.js : chatPillImage	(text, imageFile)	FormData 구성 message + image	HTTP POST 요청 multipart
4	HTTP 전송	multipart 바이너리 스트림	네트워크 전송	이미지 백엔드 도착
5	backend / routes/pill.py : chat_pill	UploadFile	이미지 파일 유효성 검증	image
6	backend / services/argos.py : analyze_pill_with_argos	image	argos에 릴레이 전달 준비	POST 요청 준비
7	HTTP 전송	multipart 바이너리	네트워크 전송 backend → argos	argos 도착
8	argos / routes/pill_analysis.py : analyze_pill	UploadFile	이미지 로드 PIL	PIL Image 객체
9	argos / services/analysis.py : run_image_analysis	PIL Image	5단계 처리 시작	내부 처리
10	argos / services/detection.py : run_detection crop 모델	PIL Image	YOLO 추론으로 알약 위치 탐지	DetectionBox 리스트
11	argos / services/images.py : PIL Image.crop	PIL Image + 박스 좌표	알약 영역만 잘라내기	잘린 PIL Image
12	argos / services/detection.py : run_detection imprint 모델	잘린 PIL Image	YOLO 추론으로 각인 위치 탐지	각인 박스 리스트
13	argos / services/ocr.py : run_ocr	각 각인 이미지	CRNN OCR로 글자 인식	각인 텍스트 리스트

14	argos / services/shape.py : run_shape_classification	잘린 PIL Image	EfficientNet 추론 + 3조건 검증	모양 라벨 또는 unknown
15	argos / services/color.py : run_color_classification	잘린 PIL Image	ResNet 추론 + softmax	색상 라벨 + confidence
16	argos / routes/pill_analysis.py : analyze_pill	각 단계 결과	PillAnalysisResponse 모델로 직렬화	JSON 응답
17	HTTP 전송	JSON	네트워크 전송 argos → backend	backend 도착
18	backend / services/argos.py : analysis_summary_from_argos	argos JSON 응답	필요한 필드 추출 후 요약본 변환	PillAnalysisSummary
19	backend / services/candidates.py : rank_candidates	PillAnalysisSummary	CSV 로드 load_csv_rows	CSV 행 리스트
19-1	backend / services/candidates.py : exact_label_score	CSV 모양 + argos 모양 라벨	완전 일치 여부 확인	0.0 또는 1.0
19-2	backend / services/candidates.py : color_label_score	CSV 색상 + argos 색상 라벨	색상 계열 정규화 후 일치 확인	0.0 또는 1.0
19-3	backend / services/candidates.py : imprint_match_score	CSV 각인 + argos 각인 텍스트	SequenceMatcher로 문자열 유사도 계산	0.0 ~ 1.0
19-4	backend / services/candidates.py : rank_candidates	3개 점수	$0.10 \times \text{shape} + 0.20 \times \text{color} + 0.70 \times \text{imprint}$	total_score
19-5	backend / services/candidates.py : rank_candidates	모든 CSV 행의 total_score	내림차순 정렬 + 상위 3개 선택	CandidateScore 리스트
20	backend / routes/pill.py : chat_pill	CandidateScore 리스트 + PillAnalysisSummary	PillChatResponse 모델로 직렬화	JSON 응답
21	HTTP 전송	JSON	네트워크 전송 backend	프론트 도착

			→ 프론트	
22	pill_chatbot_front / api/imageApi.js : chatPillImage	fetch response	JSON 파싱	PillChatResponse JS 객체
23	pill_chatbot_front / api/adapters.js : imageResponseToMessage	PillChatResponse JS 객체	메시지 객체 형식으로 변환	봇 메시지 객체
24	pill_chatbot_front / components/ChatMessage.jsx : ChatMessage	봇 메시지 객체	후보 카드 리스트로 JSX 렌더링	DOM 화면 표시

* 챗봇 흐름

단계	위치 (파일:함수)	입력	처리	출력
1	pill_chatbot_front / ChatComposer.jsx : handleSend	사용자 텍스트	텍스트 상태 저장	{ text }
2	pill_chatbot_front / ChatPage.jsx : handleUserInput	{ text }	이미지 없음 확인 → 챗봇 API 호출 결정	chatbotApi.classify 호출
3	pill_chatbot_front / api/chatbotApi.js : classify	text	JSON body 구성	HTTP POST 요청 (application/json)
4	HTTP 전송	JSON	네트워크 전송	챗봇 백엔드 도착
5	chat_backend / api/routes/classify.py : classify	ClassifyRequest (text)	파이프라인 호출	classify_pill(text) 실행
6	chat_backend / chatbot/pipeline.py : classify_pill	text (또는 parsed_override)	파이프라인 오케스트레이션 시작	내부 처리 진행
6-1	chat_backend / parser/input_parser.py : parse_input	text	Kiwi 형태소 분석 + 사전 매칭(RapidFuzz)	parsed dict { color, shape, text_marks, has_split_line }

6-2	chat_backend / classifier/stage1_text.py : stage1_filter	parsed dict	각인 글자 + 분할선 정보 기준 ChromaDB metadata 필터 조회	Stage 1 후보 리스트
6-3	chat_backend / chatbot/pipeline.py : classify_pill (되묻기 판단)	Stage 1 후보 수 + parsed	후보 수 > 50 (CANDIDATES_THRESHOLD) && 분할선 정보 없음 → needs_split_question=True	되묻기 플래그
6-4	chat_backend / classifier/stage2_shape.py : stage2_filter	Stage 1 후보 + parsed	사용자 모양이 속한 그룹(예: 원형류) 기준 필터링	Stage 2 후보 리스트
6-5	chat_backend / classifier/stage3_color.py : stage3_filter	Stage 2 후보 + parsed	색상 → Lab 좌표 변환 → 후보 간 거리 계산 → 거리 오름차순 정렬. 최상위 거리 > 35이면 needs_confirmation=True	정렬된 후보 리스트 + 확인 플래그
6-6	chat_backend / chatbot/pipeline.py : classify_pill (결과 조립)	최종 후보 + 플래그	candidates, parsed, stage_info, needs_split_question, needs_confirmation 조립	pipeline result dict
7	chat_backend / api/responses.py : build_response_from_result	pipeline result dict + user_input	플래그·후보 수 기반 응답 종류 결정 (single_result, multiple, ask_split_line, confirm_single, not_found 등)	ClassifyResponse (JSON)
8	HTTP 전송	JSON	네트워크 전송 (백엔드 → 프론트)	프론트 도착
9	pill_chatbot_front / api/chatbotApi.js : classify (응답 처리)	Fetch Response	JSON 파싱	ClassifyResponse JS 객체
10	pill_chatbot_front / api/adapters.js : chatbotResponseToMessage	ClassifyResponse 객체	kind별 메시지 변환 (botSingleResultMessage, botMultipleResultMessage, botQuestionMessage 등)	봇 메시지 객체
11	pill_chatbot_front / components/ChatMessage.jsx : ChatMessage	봇 메시지 객체	후보 카드, 상세 카드, 질문 버튼 등 JSX 렌더링	사용자 화면(DOM) 표시